

Федеральное государственное автономное образовательное учреждение высшего
образования «Национальный исследовательский университет ИТМО»

Факультет Программной инженерии и Компьютерной техники

Лабораторная работа №4
Вариант 31066

Выполнила:

Косогова Мария Александровна

Группа: Р3106

Проверил:

Вербовой Александр

Александрович,

Преподаватель практики.

Санкт-Петербург 2025

Содержание

ЗАДАНИЕ	3
ОСНОВНЫЕ ЭТАПЫ ВЫПОЛНЕНИЯ	4
1. ЗАПРОС №1	4
2. ЗАПРОС №2	6
ВЫВОД	9

Задание

psql -h pg -d ucheb

Составить запросы на языке SQL (пункты 1-2).

Для каждого запроса предложить индексы, добавление которых уменьшит время выполнения запроса (указать таблицы/атрибуты, для которых нужно добавить индексы, написать тип индекса; объяснить, почему добавление индекса будет полезным для данного запроса).

Для запросов 1-2 необходимо составить возможные планы выполнения запросов. Планы составляются на основании предположения, что в таблицах отсутствуют индексы. Из составленных планов необходимо выбрать оптимальный и объяснить свой выбор.

Изменяются ли планы при добавлении индекса и как?

Для запросов 1-2 необходимо добавить в отчет вывод команды EXPLAIN ANALYZE [запрос]

Подробные ответы на все вышеперечисленные вопросы должны присутствовать в отчете (планы выполнения запросов должны быть нарисованы, ответы на вопросы - представлены в текстовом виде).

1. Сделать запрос для получения атрибутов из указанных таблиц, применив фильтры по указанным условиям:
Таблицы: Н_ОЦЕНКИ, Н_ВЕДОМОСТИ.
Вывести атрибуты: Н_ОЦЕНКИ.ПРИМЕЧАНИЕ,
Н_ВЕДОМОСТИ.ЧЛВК_ИД.
Фильтры (AND):
а) Н_ОЦЕНКИ.ПРИМЕЧАНИЕ > отлично.
б) Н_ВЕДОМОСТИ.ДАТА > 1998-01-05.
Вид соединения: INNER JOIN.
2. Сделать запрос для получения атрибутов из указанных таблиц, применив фильтры по указанным условиям:
Таблицы: Н_ЛЮДИ, Н_ОБУЧЕНИЯ, Н_УЧЕНИКИ.
Вывести атрибуты: Н_ЛЮДИ.ИД, Н_ОБУЧЕНИЯ.ЧЛВК_ИД,
Н_УЧЕНИКИ.ИД.
Фильтры: (AND)
а) Н_ЛЮДИ.ИМЯ = Николай.
б) Н_ОБУЧЕНИЯ.НЗК = 001000.
с) Н_УЧЕНИКИ.ИД < 1.
Вид соединения: RIGHT JOIN.

План 2: Полный скан таблицы Н_ОЦЕНКИ и фильтрация по условию
 Н_ОЦЕНКИ.ПРИМЕЧАНИЕ > отлично
 Полный скан таблицы Н_ВЕДОМОСТИ и фильтрация по условию
 Н_ВЕДОМОСТИ.ДАТА > 1998-01-05
 Соединение таблиц Н_ОЦЕНКИ и Н_ВЕДОМОСТИ по условию Н_ОЦЕНКИ.КОД
 = Н_ВЕДОМОСТИ.ОЦЕНКА
 Выбираются нужные поля - Н_ОЦЕНКИ.ПРИМЕЧАНИЕ,
 Н_ВЕДОМОСТИ.ЧЛВК_ИД
 В таком плане фильтрация происходит до соединения, что значительно быстрее.

Таким образом, оптимальнее ПЛАН 2, так как он фильтрует данные до соединения, что позволяет уменьшить время выполнения запроса.

■ Индексы

```
CREATE INDEX ИНДЕКС_Н_ОЦЕНКИ_КОД ON Н_ОЦЕНКИ USING
hash(КОД)
CREATE INDEX ИНДЕКС_Н_ВЕДОМОСТИ_ОЦЕНКИ ON Н_ВЕДОМОСТИ
USING hash(ОЦЕНКИ)
CREATE INDEX ИНДЕКС_Н_ОЦЕНКИ_ПРИМЕЧАНИЕ ON Н_ОЦЕНКИ
USING btree(ПРИМЕЧАНИЕ)
CREATE INDEX ИНДЕКС_Н_ВЕДОМОСТИ_ДАТА ON Н_ВЕДОМОСТИ USING
btree(ДАТА)
```

Добавление данных индексов должно ускорить выполнение запроса:

- Во-первых, добавление индексов на Н_ОЦЕНКИ.КОД и Н_ВЕДОМОСТИ.ОЦЕНКИ ускорит выполнение соединения, так как индексы помогут быстро находить соответствующие строки для объединения (тип индекса hash подходит для оператора "=").
- Во-вторых, поскольку в фильтрах используется оператор ">", можно создать индекс btree (подходит для диапазонных запросов), что ускорит выполнение запроса.

При добавлении индексов планы выполнения запросов изменятся, так как будет происходить индексный скан и Hash Join станет быстрее благодаря индексам.

■ Вывод команды EXPLAIN ANALYZE

```
Hash Join (cost=1.15..7724.65 rows=72432 width=422) (actual time=0.065..69.518 rows=67989 loops=1)
  Hash Cond: (("Н_ВЕДОМОСТИ"."ОЦЕНКА")::text = ("Н_ОЦЕНКИ"."КОД")::text)
    -> Seq Scan on "Н_ВЕДОМОСТИ" (cost=0.00..6884.50 rows=217295 width=10) (actual time=0.010..36.918
rows=217247 loops=1)
      Filter: ("ДАТА" > '1998-01-05 00:00:00'::timestamp without time zone)
      Rows Removed by Filter: 5193
    -> Hash (cost=1.11..1.11 rows=3 width=452) (actual time=0.042..0.044 rows=2 loops=1)
      Buckets: 1024 Batches: 1 Memory Usage: 9kB
      -> Seq Scan on "Н_ОЦЕНКИ" (cost=0.00..1.11 rows=3 width=452) (actual time=0.027..0.030 rows=2
loops=1)
        Filter: (("ПРИМЕЧАНИЕ")::text > 'отлично'::text)
        Rows Removed by Filter: 7
Planning Time: 0.558 ms
Execution Time: 72.659 ms
```

2. Запрос №2

■ Реализация запроса

Таблицы: Н_ЛЮДИ, Н_ОБУЧЕНИЯ, Н_УЧЕНИКИ.

Вывести атрибуты: Н_ЛЮДИ.ИД, Н_ОБУЧЕНИЯ.ЧЛВК_ИД, Н_УЧЕНИКИ.ИД.

Фильтры: (AND)

а) Н_ЛЮДИ.ИМЯ = Николай.

б) Н_ОБУЧЕНИЯ.НЗК = 001000.

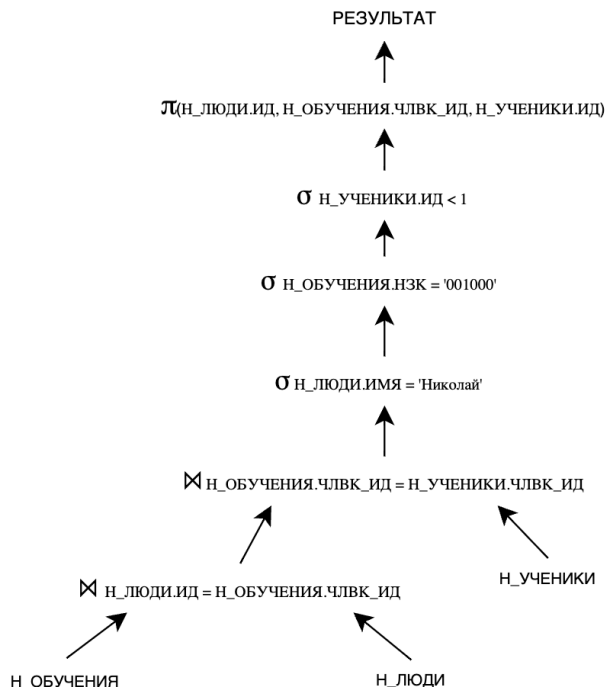
с) Н_УЧЕНИКИ.ИД < 1.

Вид соединения: RIGHT JOIN.

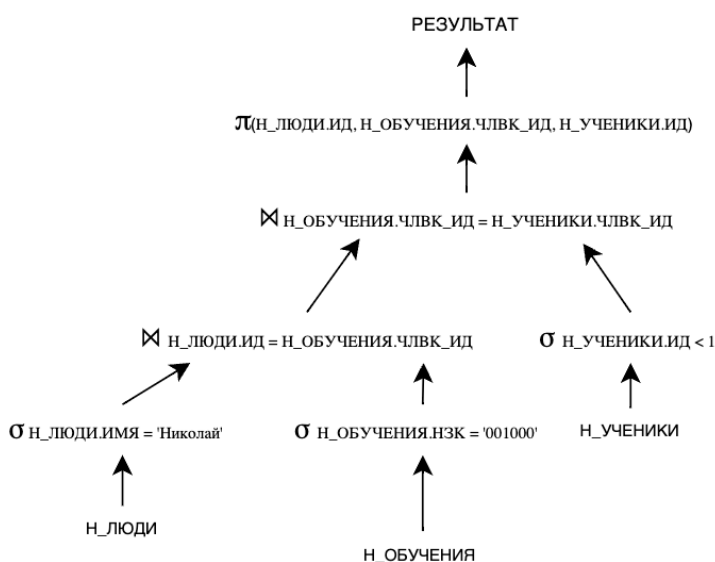
```
SELECT Н_ЛЮДИ.ИД, Н_ОБУЧЕНИЯ.ЧЛВК_ИД, Н_УЧЕНИКИ.ИД
FROM Н_ЛЮДИ
RIGHT JOIN Н_ОБУЧЕНИЯ ON Н_ЛЮДИ.ИД = Н_ОБУЧЕНИЯ.ЧЛВК_ИД
RIGHT JOIN Н_УЧЕНИКИ ON Н_ОБУЧЕНИЯ.ЧЛВК_ИД =
Н_УЧЕНИКИ.ЧЛВК_ИД
WHERE Н_ЛЮДИ.ИМЯ = 'Николай'
AND Н_ОБУЧЕНИЯ.НЗК = '001000'
AND Н_УЧЕНИКИ.ИД < 1;
```

■ Возможные планы запросов

- ПЛАН 1



- ПЛАН 2



План 1: Полный скан таблиц Н_ОБУЧЕНИЯ и Н_ЛЮДИ

Соединение таблиц Н_ОБУЧЕНИЯ и Н_ЛЮДИ по условию ИД = ЧЛВК_ИД

Полный скан таблицы Н_УЧЕНИКИ

Соединение таблиц Н_ОБУЧЕНИЯ и Н_УЧЕНИКИ по условию ЧЛВК_ИД = ЧЛВК_ИД

Фильтрация по условиям Н_ЛЮДИ.ИМЯ = Николай, Н_ОБУЧЕНИЯ.НЗК = 001000, Н_УЧЕНИКИ.ИД < 1

Выбираются нужные поля - Н_ЛЮДИ.ИД, Н_ОБУЧЕНИЯ.ЧЛВК_ИД, Н_УЧЕНИКИ.ИД

В таком плане система обрабатывает все пары строк, даже те, которые позже отсеются (сначала происходит соединение, а потом фильтрация), что значительно медленнее.

План 2: Полный скан таблицы Н_ОБУЧЕНИЯ и фильтрация по условию Н_ОБУЧЕНИЯ.НЗК = 001000

Полный скан таблицы Н_ЛЮДИ и фильтрация по условию Н_ЛЮДИ.ИМЯ = Николай

Соединение таблиц Н_ОБУЧЕНИЯ и Н_ЛЮДИ по условию ИД = ЧЛВК_ИД

Полный скан таблицы Н_УЧЕНИКИ и фильтрация по условию Н_УЧЕНИКИ.ИД < 1

Соединение таблиц Н_ОБУЧЕНИЯ и Н_УЧЕНИКИ по условию ЧЛВК_ИД = ЧЛВК_ИД

Выбираются нужные поля - Н_ЛЮДИ.ИД, Н_ОБУЧЕНИЯ.ЧЛВК_ИД, Н_УЧЕНИКИ.ИД

В таком плане фильтрация происходит до соединения, что значительно быстрее.

Таким образом, оптимальнее ПЛАН 2, так как он фильтрует данные до соединения, что позволяет уменьшить время выполнения запроса.

■ Индексы

```
CREATE INDEX ИНДЕКС_Н_ЛЮДИ_ИД ON Н_ЛЮДИ USING hash(ИД)
CREATE INDEX ИНДЕКС_Н_ОБУЧЕНИЯ_ЧЛВК_ИД ON Н_ОБУЧЕНИЯ USING
hash(ЧЛВК_ИД)
CREATE INDEX ИНДЕКС_Н_УЧЕНИКИ_ЧЛВК_ИД ON Н_УЧЕНИКИ USING
btree(ЧЛВК_ИД)
CREATE INDEX ИНДЕКС_Н_ЛЮДИ_ИМЯ ON Н_ЛЮДИ USING hash(ИМЯ)
CREATE INDEX ИНДЕКС_Н_ОБУЧЕНИЯ_НЗК ON Н_ОБУЧЕНИЯ USING
hash(НЗК)
CREATE INDEX ИНДЕКС_Н_УЧЕНИКИ_ИД ON Н_УЧЕНИКИ USING btree(ИД)
```

Добавление данных индексов должно ускорить выполнение запроса:

- Во-первых, добавление индексов на Н_ЛЮДИ.ИД и Н_ОБУЧЕНИЯ.ЧЛВК, Н_УЧЕНИКИ.ЧЛВК_ИД ускорит выполнение соединения, так как индексы помогут быстро находить соответствующие строки для объединения (тип индекса hash подходит для оператора "=").
- Во-вторых, поскольку в фильтре Н_УЧЕНИКИ.ИД < 1 используется оператор "<", можно создать индекс btree (подходит для диапазонных запросов), что ускорит выполнение запроса.
- В-третьих, поскольку в фильтрах Н_ЛЮДИ.ИМЯ = Николай и Н_ОБУЧЕНИЯ.НЗК = 001000 используется оператор "=", можно создать индекс hash (подходит для оператора "="), что ускорит выполнение запроса.

При добавлении индексов планы выполнения запросов изменятся, так как будет происходить индексный скан и Nested Loop Join станет быстрее благодаря индексам.

- Вывод команды EXPLAIN ANALYZE

```
Nested Loop (cost=0.85..20.15 rows=1 width=12) (actual time=0.009..0.010 rows=0 loops=1)
-> Nested Loop (cost=0.57..16.54 rows=1 width=12) (actual time=0.008..0.009 rows=0 loops=1)
    -> Index Scan using "УЧЕН_PK" on "Н_УЧЕНИКИ" (cost=0.29..8.02 rows=1 width=8)
        (actual time=0.008..0.008 rows=0 loops=1)
            Index Cond: ("ИД" < 1)
    -> Index Scan using "ЧЛВК_PK" on "Н_ЛЮДИ" (cost=0.28..8.30 rows=1 width=4) (never
        executed)
            Index Cond: ("ИД" = "Н_УЧЕНИКИ"."ЧЛВК_ИД")
            Filter: (("ИМЯ")::text = 'Николай'::text)
    -> Index Scan using "ОБУЧ_ЧЛВК_FK_I" on "Н_ОБУЧЕНИЯ" (cost=0.28..3.60 rows=1
        width=4) (never executed)
        Index Cond: ("ЧЛВК_ИД" = "Н_ЛЮДИ"."ИД")
        Filter: (("НЗК")::text = '001000'::text)
Planning Time: 0.730 ms
Execution Time: 0.060 ms
```

Вывод

Выполнение лабораторной работы по базам данных позволило мне более углубленно познакомиться с декларативным языком программирования SQL. Данная работа дала мне возможность применить полученные теоретические знания на практике, развить навыки работы с информацией. В процессе выполнения я оптимизировать запросы, составлять наиболее выгодный план выполнения запросов, используя для этого подходящие виды индексов. Приобретённый опыт поможет мне при дальнейшем изучении предмета базы данных и языка SQL.